ARC-AGI-3 Part 2: Engineering and Debugging Log (Plain-Language Edition)

Bo Hou (compiled with Claude)

2026-10

0 What this note covers

Part 1 covered what the task is, how to formalize a two-layer world model, and what others have done. This note picks up from there and covers what happened **once we actually started building**: how the code changed version by version, which pitfalls each debugging round hit, why each change was made, and which trade-offs we discussed along the way.

No math derivations here (GDN has its own side story). This is all engineering: what was slow, what blew up, and where the bugs were our own.

Reading guide:

- "**Bo's call**" = direction and rules, decided by Bo.
- "**Claude's optimization**" = implementation-level changes Claude added on its own; each one says *why*.
- Every number is measured or simulated and says which. The real GPU run has not started yet, so some numbers come only from a simulator.

1 Starting point: what we are building

1.1 The goal in one sentence

Have an **offline** ~30B open model (Qwen) play games it has never seen: look at the frame → guess the rules → write the rules as Python code (a world model) → search for an action sequence in that code on CPU → execute. After each game, useful code is distilled into a shared library for the next game.

Mapping to the two layers in Part 1:

- **Inner layer** (one per game): the `Model` class Qwen writes (predicts the next frame) + `plan` (searches for actions) + a goal hypothesis `is_win`.
- **Outer layer** (shared across games): the model itself + the shared library `lib` + prompts and pipeline.

1.2 Bo's rules (unchanged throughout)

1. **The source code of the 25 public games may not be learned from**: “that code is for testing and maybe future RL — no, not even RL; it is the answer key, not a label, otherwise what is there to learn”. Not for training either: “not that either; using it would be cheating”.
2. Only Bo and Claude may compare generated code against the source, only on the 12 dev games, and only to improve generic prompts. Qwen never sees source code, game names or human baselines.
3. The other 13 games are a held-out set that **nobody looks at**.
4. Synthetic games may only be written by an open-weights model (not ChatGPT, for terms-of-service reasons).
5. Offline ~30B models only; no “big model as teacher” route (too much engineering).
6. Design principles: “**If a program can run it, don’t reason about it in natural language; if it can be formalized into a program, formalize it**”; “**if an outer-layer program doesn’t work, remove it**”.
7. All levels of a game share **one conversation**; thinking length must be bounded (the offline competition has a time limit).
8. The GPU machine (RunPod) may not be stopped, deleted or spent on without Bo’s approval.

1.3 Hardware

Setting	Hardware	Notes
Kaggle competition	1× RTX PRO 6000 (96 GB), offline, 9 hours total	110 hidden games run together
Our experiments	RunPod A100 80 GB	persistent volume /workspace, vLLM server

2 Step one: mining rules from human play logs

2.1 What we did

Before writing the harness, we ran statistics on **human players’ play logs** (what was pressed at each step and how the frame changed — not game source) to see what habits people share when facing an unfamiliar game. The aim was to turn the reliable habits into a few lines of “advice” in Qwen’s prompt, as a prior.

Six questions in total (all numbers are medians across games):

ID	Question	Result (plain words)
R1	How fast do people try every action?	Actions 1–6 are usually first used within the first 3–9 steps, and almost never go unused for

ID	Question	Result (plain words)
		a whole game; action 7 (undo) comes very late and is never used in about 70% of games
R2	Do clicks land on objects, and do they change something?	First computed as nearly 0 (a stats bug, see below); after the fix 0.86–1.0: people click objects, and things change near the click
R3	If an action did nothing, do people press it again?	Yes, but almost always in vain: in the recheck, out of 4,210 such “press again after no effect” cases, the frame actually changed only about 1% of the time
R4	If an action did something, do people keep pressing it?	A bit over half the time (≈ 0.5–0.65) — effective actions usually need to be repeated
R5	After game over, do people restart right away?	About 1/3
R6	When do people stop to think?	Relative to a normal step: about 1.6× before using an action for the first time, **about 6× after clearing a level**

There was also an R7; Bo judged it useless and it was dropped.

2.2 The energy-bar detector

Many games have a thin bar at the edge of the screen: it shrinks by one cell with every action, and the game is over when it runs out. It is a **step / energy counter** (part of the game interface, the HUD), not a game object. It causes two problems:

- When counting “did this step have an effect”, the bar changes on every step, so almost every step counts as “effective” and statistics like R3 get polluted;
- Later, when Qwen writes a world model, if it doesn’t know this is a counter it will treat it as an object to understand and control, which is wasted effort.

So we needed a program that finds it **from the frames alone**. Three approaches were tried:

Method	How it searches	Result
Frequency	In human logs, any pixel that changes in more than 80% of steps is HUD	**Found no pixels at all** (0 in every game). Reason: the bar changes only one or two pixels

Method	How it searches	Result
		per step, at a different position each time, so each single pixel changes rarely
D1 shape	First frame only: a same-color strip near the edge (within 6 px), thin (≤3), long (≥8)	Median precision and recall both 1.0; found all 22 games that have a HUD. Errors: also reported a HUD in the 3 games that have none; in 2 games the HUD is not a thin strip and was almost entirely missed
D2 shape + change check	D1 candidates, additionally required to change slightly in the first few steps	**Worse**: found only 12 of 22; median pixel recall fell to 0.02. Reason: in the first few steps the bar often hasn't moved yet

How we knew what was right: the game engine draws the "interface layer" (HUD) separately; we used that as the answer key **only to score the detectors**. The detectors themselves only see frames, and Qwen never sees the answer key.

What went into the harness is the shape-plus-online-confirmation idea (see the v6 perception module): a thin strip within 3 px of the edge that changes by only a few pixels (≤4) on most steps is treated as a counter, reported as a single `counter` line per step instead of as object changes. The difference from D2: it doesn't decide in the first few steps, it accumulates evidence while playing.

2.3 Only four lines made it into the prompt

1. Early on, try every available action once (undo can wait). — from R1
2. For clicks, click objects first, one click per object; empty space later. — from R2
3. If an action had no effect (counter changes don't count), don't repeat it in the same situation. — from R3
4. A thin bar at the edge that changes a little on every step is usually a step / energy counter: a limited resource, not a game object. — from the energy-bar detector (2.2)

R4–R6 only describe human behavior and don't directly help "use fewer steps", so they were left out. R6 (people pause long after a level) later shows up indirectly in "let the model decide whether this step needs deep thinking".

2.4 Lessons

1. **When a number looks wrong, check your own code first.** R2 near 0 is obviously implausible (clicking something almost always changes it). It turned out to be Claude's stats bug: click coordinates are `(x, y)` = (column, row), but the stats treated them as

(row, column), so it was looking at the wrong spot. Bo: “As expected, I still have to review it myself.” The rule since then: **verify before concluding**.

2. **Define “no change” carefully first.** A counter changes on every step; without removing it, almost every step counts as “effective” and R3-style stats are wrong. Hence the energy-bar detector (2.2), which later became the automatic counter detection in the v6 perception module.
3. **Score detectors against an answer key, not intuition.** The frequency method looked reasonable, and scoring showed it found nothing; D2 added a confirmation step that intuitively should help, and scoring showed it made things worse.
4. **Look at raw counts in rechecks, not just ratios.** The first R3 result was a single ratio that couldn’t tell whether “pressing again” ever helps; the recheck counted 4,210 cases with only 1% changing anything, and only then did the conclusion hold.
5. **Medians across games hide differences.** Some games are click-only, some have only arrow keys; per-game tables differ a lot, so the medians are only rough advice, which is why the prompt says “advice, not law”.
6. **Human rules are shallow.** Bo’s judgment: “the rules we got from humans feel pretty shallow”. They help avoid a few wasted moves early on; what really decides success is the “search success rate” — the model writes correct code, and the CPU search finds a winning path. So all effort afterwards went into the harness.

3 Round 1: the first run with a 27B dense model

3.1 The pipeline at the time

At every step the frame, changes and object list were packed into one big prompt for Qwen3.8-27B, which replied with actions or with `Model` code. Once code was written:

- **fit**: a program replays the model over every recorded step and computes prediction accuracy (Claude’s design: a program decides whether the model is right, not the LLM).
- Sample k models per round (4 in Round 1) and **keep the one with the best fit score**. How the k copies differ: the same prompt is sent as k simultaneous requests, and each token is drawn from a probability distribution (temperature 0.6, top_p 0.95, top_k 20), so each copy of the code is different; no random seed is set and the prompt is not varied. By v6.1 the default is 3, adjusted automatically by how many games are playing at once, up to 6.
- **plan**: Qwen’s search code runs in a subprocess and returns an action sequence, executed one step at a time with each step checked against the prediction; on a mismatch it stops.
- After a game, Qwen refactors the shared library.

3.2 Overnight result: 5 of 12 games scored 0, timeouts everywhere

Bo asked Claude to judge whether it was “a code problem or a design problem”. Broken down, most of it was **on our side**:

Symptom	Cause	Fix
The model thinks for a very long time on every step	The thinking-level parameter was put in the wrong place (see below the table); the 27B server **silently ignored it**, so it always thought at the highest level	Put it in the right place
Many timeouts and retries	600 s client timeout + automatic retry, so one long think was rerun several times	Timeout raised to 1800 s, no retry on timeout
ft09’s model guessed the wrong goal	**Claude’s bug**: the harness passed only the frames before and after an action and **dropped the animation frames** in between. ft09’s hint appears only in the animation, so to Qwen “nothing changed”	Keep animation frames
Shared library grew from 75 to 1,085 lines	Every game added code to the library, useful or not, correct or not	A gate (see below)
A level had only model.py and no plan	Thinking too long, context too large; time ran out before planning	See the next chapter

What “the parameter was in the wrong place” means. Every time the program asks the model something, it sends a JSON request to the vLLM server. The same parameter can go in two places:

```
{ "model": "...", "messages": [...],
  "reasoning_effort": "high",                          # top level
  "chat_template_kwargs": {"reasoning_effort": "high"} # inner level
}
```

- **Top level**: next to model and messages. This is the standard OpenAI API way. But this vLLM server accepts it **without an error and without using it** — as if it weren’t there.
- **Inner level**: inside chat_template_kwargs. These parameters go to Qwen’s “chat template” (the program that turns the conversation into the text the model actually sees); the template writes the thinking level in, and only then does the model follow it.

We had put it at the top level, so whatever we set, the model thought at its default highest level and took a long time on every step. Moving it to the inner level fixed this (and showed along the way that 27B doesn't accept the level "high"; it returns a 400 error).

Claude's optimization: a gate for the shared library. Why: the library is the outer layer; the more junk it holds, the longer every game's prompt gets and the more easily the model is led astray. Rules:

- Only code with fit ≥ 0.8 (a verified model) may enter the library;
- At most 200 lines added at a time; the updated library must import successfully;
- Functions unused for 3 games in a row are deleted (a few core ones excepted) — exactly Bo's "if an outer-layer program doesn't work, remove it";
- The prompt shows only function signatures, not bodies. Why: saves tokens; Qwen can just call them when needed.

3.3 Two discussions

"Using a sledgehammer to crack a nut." Bo pointed out: why have an LLM reason in natural language about something a program can run, like BFS? Compute must be allocated carefully; whatever can go to the CPU should. So it was settled: the LLM only "thinks up rules and writes code"; search, verification and statistics all go to the CPU.

"Let the AI decide how much compute it needs." Claude added `next: fast/think`: in each reply Qwen says whether the next step needs deep thinking. Why: simple steps (e.g. following a plan) need no thinking, and the time saved goes to hard steps.

4 Switching to a MoE model, and "thinking too long gets cut off"

4.1 Why switch

Model	Single stream	4 concurrent
Qwen3.8-27B (dense)	≈ 28 tokens/s	—
Qwen3.6-35B-A3B-FP8 (MoE, only ~3B active per token)	≈ 155 tokens/s	≈ 232 tokens/s

More than 5× faster. Bo's judgment: "if the code is about the same, go MoE; I don't think it'll be much worse".

4.2 MoE smoke test: 20 of 23 calls truncated

The MoE model has only an on/off switch for thinking, no thinking levels. Once thinking is on it doesn't stop, and when the 8,192-token limit is hit **it gets cut off before it has started**

writing the answer — all that thinking is wasted.

Claude's optimization: a thinking budget + "rescue". The thinking limit is set to 4,096; if it gets cut off, the last part of what it has already thought is taken and sent in a **non-thinking** request: "this is where your thinking got to; now give the answer directly". Why: the thinking already spent isn't wasted, and it doesn't have to think again. In a simulated test, all 110 truncations were rescued.

4.3 Bo's question: how do others handle overly long thinking?

After a survey: approaches that can afford frontier APIs don't care (thinking can be as long as it likes); offline approaches (e.g. Franzen) use **multi-turn conversation + prefix caching**, thinking only a little each turn and building up through the conversation history. That is the path we took from here.

5 v4: one conversation per game + asynchronous CPU plans

5.1 The root problem Bo pointed out

At the time every step **opened a new conversation** and rebuilt the full prompt. Bo: "Are we opening a new one at every step? For every step of a game? That's clearly wrong… the different levels of one game should definitely share one conversation."

And another point: "while the LLM is, say, running a program, it can release the GPU and come back after 30 seconds to look at the result, instead of hanging there waiting for the CPU."

5.2 What v4 does

- One game = one conversation; all levels continue the same chat. Each new message carries only **what is new** (what happened, new steps, current frame).
- When Qwen replies with a `plan` code block, the program runs it on CPU in the background while **the GPU serves other games**; after 30 seconds it asks Qwen: "still running, here's the latest output — wait or switch?" If Qwen replies `wait` it keeps waiting; anything else kills the plan.
- When the conversation gets too long, a few of the earliest turns are deleted.

5.3 v4 smoke test (ar25, 20 minutes): 33 of 59 calls errored

All errors were vLLM returning 400: "context too long" (≥ 127k tokens). The cause: **Claude's token-count estimate was wrong**. It estimated "characters ÷ 3", but our frames are strings of hex digits and Qwen's tokenizer makes **each digit one token**, so the real count was about 3× the estimate. The program thought the context was still short when it had long since overflowed.

The good news is that it still moved forward between errors: 106 actions, 5 model versions, the last of which predicted 100% of recorded steps correctly, and a 55-step plan executed. But it never wrote the `goal:` line (its goal hypothesis).

Claude's fix: a conservative estimate (about 1.6 characters per token) as a stopgap, plus a fallback — if the server does report "too long", delete a few of the earliest turns and retry.

6 v5: frames no longer pile up in the conversation

v4 worked, but dozens of 64×64 frames piled up in the conversation, each over a thousand tokens, and it filled up fast.

Claude's optimizations (v5):

1. **Keep only the current frame in the conversation.** When the next message arrives, the previous frame is replaced by a one-line placeholder. The program keeps every frame; to see an old one Qwen writes a `look` code block (Python run on CPU, e.g. `print(grid(frames[3], 0, 15, 20, 40))`). Why: frames take the most space, and Qwen almost always needs only the current one.
2. **Calibrate the token estimate with the server's real numbers.** Every response includes the real `prompt_tokens`; the program derives a ratio from it (digits count as 1 token each, other characters at the calibrated rate) and keeps a 4–8% margin. Why: the lesson from v4 was "off by 3×", so instead of guessing, use the server's true value.
3. **When too long, cut one big block at once**, and cut the real conversation, not a temporary copy (v4 only cut the copy, so it overflowed again next time).
4. **Sandbox.** Qwen's code runs as a separate low-privilege user with a memory cap, a CPU-time cap and its own process group (killing it also kills its children); plus a static check: code containing `import os`, `subprocess`, `open(` and the like is not run, and Qwen is told why. Why: Qwen's code is untrusted; deleting files or hanging the machine are both possible.
5. **One game crashing doesn't affect the others**: the exception goes to error.txt and the batch continues.

Several rounds of "reviewer" agents also checked the code (only ever allowed to touch dev games).

A violation to state plainly: once a reviewer agent ran a random-action test on the held-out game bp35. It did not look at source or change anything, but it violated "nobody looks at the held-out set". Afterwards the check script got a hard limit — it refuses to run on non-dev games — and reviewer prompts hard-code which games may be touched.

7 v6: more “human” step descriptions, fuller scheduling

7.1 Describe changes in objects, not pixels (percept.py)

Before, Qwen was told “which pixels changed”. The problem is that Qwen then has to assemble a pile of pixel coordinates into “which thing moved” — exactly what Bo meant by “if a program can run it, don’t make the LLM reason about it”.

Claude’s optimization: a CPU perception module. The program splits the frame into same-color connected regions (objects), compares the two frames, and says directly:

- `c9 3x3 @(10,12) moved (+0,+3)` (a blue 3×3 block moved 3 cells right)
- recolored / grew / shrank / new / gone
- the thin edge bar that shrinks every step (step counter) is detected automatically and reported as a single `counter`, no more flooding every step;
- returning to a frame seen before is reported as “same as frames[k]”.

The program also keeps an **action-effect table**: what each action has done so far (e.g. “action 1: mostly moves some object up by 1”). Why: these are deterministic computations, fast and exact on CPU; measured at 2.6–3.6 ms per step.

Representation of the same frame	Approx. tokens
Full hex frame	870–2,060
Object list	150–400

7.2 Other token savings

- **Collapse repeated rows**: rows identical to the one above are written as `a..b same as row a-1`. Frames with large uniform backgrounds shrink a lot.
- **Don’t resend an unchanged frame.**
- **Model check reports show only what changed**: errors already reported are not repeated; only newly wrong steps are listed, explained in terms of objects.

7.3 Scheduling

Claude’s optimization: rolling scheduling. Before, a batch of 4 games started and ended together, and the slowest one held up the whole batch. Now **4 games are always in play**; when one finishes another starts immediately, and the post-game “refactor the shared library” runs on a background thread without taking a game slot. Why: the GPU never sits idle.

Slow-fit throttling: if fitting one model takes very long (Qwen wrote slow code), fewer copies are sampled.

Timing: each game records where its time went: LLM / fit / look / plan / game environment. Why: once on the GPU, find the bottleneck first, then optimize — no guessing.

8 v6.1: the prefix-cache trap (the biggest debugging episode of this period)

8.1 What a prefix cache is (plain words)

In a multi-turn conversation, each prompt sent to the server = everything before + one new message. The long earlier part was already computed last time; the server stores the computed results (the KV) and reuses them, computing only the newly added bit. That is the **prefix cache**. The condition: **the earlier part must be exactly identical**.

In an ordinary model the KV is stored in blocks of 16 tokens, and each block gets a fingerprint (a hash, computed on CPU) from "the previous block's fingerprint + these 16 tokens". A new request matches block by block, and matching blocks are reused directly. Even if a block in the middle changed, the blocks **before the change** can still be reused.

8.2 Qwen3.6 is different

Qwen3.6 is a **hybrid model**: about 3/4 of its layers are linear attention (GDN, the one derived in the side story), only 1/4 are ordinary attention. Linear-attention layers don't store a KV per token; they store one **fixed-size state** (in effect, everything so far compressed into one matrix).

Here's the problem: that state can only be saved as a "snapshot" at some position, and vLLM keeps only **one** snapshot per request, at a position aligned to a few hundred to a few thousand tokens. Reuse requires **everything from the start up to the snapshot position to be unchanged** (vLLM issue #45238).

And what v5/v6 did was "replace the previous frame with a placeholder every turn" — **every turn edited the previous message**, the content before the snapshot changed, so **the cache was invalidated every turn** and the whole conversation was recomputed from scratch. The longer the conversation, the bigger the loss.

Bo's question in the middle hit the point: "if images are only dropped after six, the prompt S shouldn't change every time" — right, the problem was that v6 dropped **every turn** (eager), so it changed every turn.

8.3 Claude's optimization: lazy mode + a startup probe

- **lazy**: the conversation only grows at the end; nothing earlier is edited. Old frames stay for now; once there are more than 6 (or the conversation is too long), all old frames, old

tables and old images are cleared **at once**. Why: one cleanup breaks the cache once, and then many turns reuse it in a row; eager breaks it every turn.
- **eager**: delete every turn, shortest conversation. Suits ordinary models whose cache tolerates edits in the middle, or no cache at all.
- **Automatic startup probe**: at startup the program sends 4 test requests to the server, measures cache hits for “append only at the end” and “edit in the middle”, picks lazy or eager automatically, and writes the result to PROBE.txt. Why: different vLLM versions behave differently; don’t guess.

Results on a simulator (written to follow the hybrid model’s cache rules):

Mode	Share of prompt tokens served from cache
eager (v6)	≈ 50–55%
lazy (v6.1)	≈ 85%

In other words, the input tokens the GPU actually has to compute drop by about 2/3.

8.4 Other changes in the same version (all Claude’s)

- **Growing wait intervals**: while a plan runs on CPU, the first check is after 30 seconds, then the interval doubles (up to 4×), never past the deadline. Why: each check costs a GPU inference; asking 10 times about a 5-minute search is waste.
- **Adaptive number of candidates**: when few games are playing, sample more models each time (up to 6); when many are, sample fewer. Why: an idle GPU is wasted anyway.
- **Image limit**: by default vLLM allows at most 16 images per request and errors beyond that (this killed games for JustAdev). The program defaults to at most 8; if the server says “at most N”, it learns that limit and drops old images; for other image errors it first drops the newest image, then all of them, and after 3 errors in a row turns images off.
- **No MTP (multi-token prediction speedup)**: in vLLM, prefix caching + MTP corrupts the output (vLLM issue #53912). Only one can be had; we chose the cache.
- The log records cache hits per call and where the cache broke, for checking once on the GPU.

Another review pass found 9 small issues (e.g. only one image dropped per turn when over the limit, the wait interval could pass the deadline, cache shown as 0% when the server returns no cache details), all fixed. All 28 simulated scenarios pass.

9 Images: should Qwen see pictures?

This was the longest discussion; here it is in order.

9.1 Bo’s starting point

A 64×64 frame is easy for a human eye, and a program can directly give every object's position and color. So does Qwen really need an image? Whatever a few variables can describe needs no image. Minimum inference cost and minimum context use are the goal.

9.2 Cost comparison (same frame)

Form given to Qwen	Approx. tokens	Who does the work
Object list (computed by program)	150–400	CPU, 2.6–3.6 ms
Hex text frame	870–2,060	almost nothing
Image (upscaled to 640×640)	≈ 400	GPU vision encoder

Why upscale: Qwen's vision encoder cuts the image into patches of 14–16 pixels; in the raw 64×64 image one cell is a single pixel, so several cells fall into the same patch and blur together. After upscaling, a cell is 10×10 pixels and can be told apart. And the token count is set by **the pixel size sent in** (the new encoder is roughly 32×32 pixels = 1 token, so 640×640 ≈ 400); the model cannot upscale internally on its own.

9.3 Bo's "ideal design"

The raw 64×64 image takes only about 10 tokens in the context; upscaling to 640×640 and further image processing all happen in the model's **internal** computation and have nothing to do with the context.

Claude searched: models that compress an image into very few tokens do exist (e.g. 1 or 2 tokens per image), but they all "shrink a big image" to save tokens and lose detail; what we need is "**a small image, every pixel exact**". Nothing ready-made that plugs into Qwen3.6 was found. This could be a future RL / paper direction.

9.4 What others do (from their open-source code)

Approach	How images are given
Franzen (top offline tier)	Image every turn, nearest-neighbor upscaled **10×**; he measured that **going from 4× to 10× was the biggest gain images brought**; also gives "change images" and game-over frames; images stay in the history (append-only, which suits the prefix cache)
JustAdev	4× upscale, at most 12 images (exceeding vLLM's 16-image limit had killed games)
Retrodict	Text everywhere; only at the start does a separate vision model look at the image once (12× upscale)

9.5 Current approach (local version, not yet uploaded)

Claude's implementation: one image per frame + a change image.

- When the frame changes, attach an image of the current frame (the 10× idea, capped at 1280×640);
- When something changed within the same level, also attach a "change image": changed cells in their own color, unchanged cells painted dark navy (a color not in the palette, so no confusion);
- Works with lazy mode: images are only appended and cleared all at once when there are too many.

Why: Franzen's measurements are more reliable than our intuition; the text frame and object list are still given, and the image is an extra channel. Whether it helps will be settled by A/B tests once the GPU runs (no images / images on request / image every frame / different upscale factors).

10 Notes on how we work together

- **Bo felt "throughput was low"**: Bo deliberately pasted intermediate steps back and forth to understand what Claude was actually doing — Claude works fast, but to Bo it felt like a black box that was easy to lose track of. That is a normal cost, and this note is meant to open the black box.
- **Claude's mistakes**: the R2 stats bug, the dropped animation frames, the 3× token estimate, the reviewer agent touching a held-out game — all on Claude's side, caught in Bo's reviews or found afterwards.
- **The rules were always Bo's**: data boundaries (which games may be looked at), one conversation per game, don't give the LLM what the CPU can do, delete outer-layer code that doesn't work, don't touch the GPU machine without asking.

11 Where we are and what's next

Status: v6.1 is on Google Drive (a local version that adds per-frame images awaits Bo's decision on uploading). All simulated tests pass. RunPod is still waiting for an A100.

Questions only the GPU can answer: the vLLM version and its real cache behavior (see PROBE.txt), how many tokens an image really costs, whether the sandbox can set permissions on the network volume, whether Qwen follows the reply format, and real throughput.

Next:

1. First real run: 4 games in parallel; after 15 minutes check the cache hit rate and where time goes;
2. Image A/B tests;

3. Adjust prompts based on the results, then move toward RL (GRPO, with the group mean as baseline);
4. Write Part 3 / the paper.

Appendix: versions at a glance

Version	Main changes	Problem solved
Round 1	One big prompt per step; pick models by fit; plan in a subprocess	Get the pipeline running
v3	Switch to MoE; thinking 4,096 + rescue	Too slow; thinking cut off
v4	One conversation per game; async CPU plan + wait	New conversation every step; GPU idle-waiting
v5	Keep only the current frame; real token calibration; sandbox	Context overflow (estimate off by 3×)
v6	Object-level descriptions; effect table; rolling scheduling; timing	LLM doing a program's job; idle GPU
v6.1	Lazy deletion + startup probe; wait backoff; image limits	Hybrid-model cache invalidated every turn
Local	Per-frame image + change image	Image evidence (Franzen 10×)

Appendix: links

- vLLM issue #45238 (hybrid-model prefix cache is reused only if nothing before the snapshot changed): https://github.com/vllm-project/vllm/issues/45238
- vLLM issue #53912 (prefix cache + MTP corrupts output on hybrid models): https://github.com/vllm-project/vllm/issues/53912
- Franzen (image every turn, 10× upscale): https://github.com/tonghuikang/daniel-franzen-arc-agi-3
- JustAdev (4× upscale, at most 12 images): https://github.com/JustAdev742/Arc-Agi-3-Kaggle-comp
- Retrodict (text-first, a vision model looks once at the start): https://github.com/ryanbbrown/Retrodict